Review Report

Project: PawSafe

Reviewer: HIMM (Role: Auditor)

1. Methodology Under Review

3-Tier Layered Architecture Sistem: Flutter tabanlı Presentation Layer, iş kurallarını barındıran Repository + Cloud Functions Layer ve veri katmanı olarak Cloud Firestore üzerine inşa edilmiştir.

2. Technological Context

Projenin teknolojik bağlamı, gerçek zamanlı veri akışını gerektiren harita odaklı bir sistemdir. Bu akışı yönetmek için Riverpod ile reaktif durum yönetimi (state management) benimsenmiştir. Mimari kararlarda filtreleme için Strategy, konum servisleri için Singleton tasarım örüntüleri tercih edilmiştir .

3. Summary of the Method

Mimari yöntem, kullanıcı arayüzü ile veri tabanı arasına net bir "Repository" soyutlaması koymaya dayanır. UI katmanı iş mantığı barındırmaz; Riverpod aracılığıyla Repository'leri dinler ve veritabanındaki (Firebase) değişiklikler anlık olarak haritaya yansır.

4. System Evaluation

Sistem, "Auditor" perspektifinden olağanüstü derecede iyi ve izlenebilir (traceable) tanımlanmıştır. Her bir kullanım senaryosu (Use Case), spesifik Fonksiyonel Gereksinimlere (FR) bağlanmış ve bunlar UML Sequence diyagramlarıyla detaylandırılmıştır. Roller, mimari tasarım ve tasarım örüntülerinin neden seçildiği mantıksal olarak çok net temellendirilmiştir.

5. Assumption Validity

Geçerli: Pil tüketimini ve eşzamanlı donanım çağrılarını önlemek için GPS servisinin Singleton olarak tasarlanması (tek bir oturum açılması) donanım verimliliği açısından son derece geçerli bir varsayımdır .

Geçersiz: Ekibin %25'lik eşit iş dağılımıyla "herkesin her şeye dokunacağı" varsayımı, katı bir versiyon kontrol (Git) stratejisi belirlenmediği için geçersiz kalmıştır. Bu durum karmaşık ve düzensiz commit mesajlarına yol açmaktadır.

6. Identified Weaknesses

Tasarım kağıt üzerinde mükemmel olsa da, yürütme (execution) sürecinde zorlanmalar mevcuttur. Ekip tam teşekküllü bir mobil uygulama derlemeye (deployment) çalışarak gereksiz bir kurulum karmaşıklığı yaratmış ve asıl mimariyi test etme odağından uzaklaşmıştır. Ayrıca, işbölümü pratiklerindeki zayıflık nedeniyle GitHub commit geçmişi projenin kurumsal mimari iddiasını zedeleyecek kadar düzensizdir.

7. Missing Mechanisms

- "Herkesin her katmanda çalışması" kararını destekleyecek bir Git Branching Strategy (Dallanma Stratejisi) ve Conventional Commits (Standartlaştırılmış Commit Mesajları) mekanizması eksiktir.

- Tüm ortamı (Android/iOS) ayağa kaldırmadan, mimarinin temel mantığını test edebilecekleri bir **Local Prototyping (Yerel Prototipleme)** stratejisi eksiktir.

8. Scenario Evaluation

Senaryo: Kullanıcı haritada sadece "Son 1 saat" içindeki raporları görmek için filtre uygular.

Davranış: Sistem bu karmaşıklığı *Strategy Pattern* ile yönetir . Uzun ve karmaşık `if/else` blokları yazmak yerine, Riverpod (MapNotifier) üzerinden `HourFilter` stratejisi tetiklenir ve yeni kurallar Repository'e aktarılır . Seçilen tasarım örüntüsü, sistemin gereksinimleriyle mükemmel bir uyum (alignment) içindedir.

9. Improvement Assessment

Dağıtım (deployment) sorunlarını aşmak için ekibin derhal canlı bir mobil uygulama çıkarma hedefinden vazgeçip, uygulamanın çekirdek işlevlerini gösteren **localde çalışan bir prototip** geliştirmeye yönelmesi harika bir çözüm olacaktır. Bu sayede mimarinin (Strategy, Singleton, Repository) gerçekten çalışıp çalışmadığı kanıtlanabilir. Ayrıca commit mesajları için bir standart (örneğin GitFlow) belirlenmesi proje yönetimini toparlayacaktır.

10. Implementation Assessment

Küçük bir prototip yapma fikri, ekibin şu anki karmaşık derleme/uygulama sorunlarını (implementation block) kesinlikle aşmasını sağlayacaktır. Tasarım dokümanlarındaki yüksek mühendislik kalitesi, şu anki düzensiz kodlama ve commit süreci tarafından desteklenmemektedir; prototipe geçiş bu uyumsuzluğu giderecektir.

11. Overall Strengths

Mükemmel izlenebilirlik (Traceability). Alınan mimari kararların (Örn: Flutter vs React Native veya neden Strategy kullanıldığı) sadece kod seviyesinde değil, Kalite Nitelikleri (Quality Attributes) seviyesinde gerekçelendirilmesi. Tasarımda gereksiz karmaşıklıktan (overengineering) kaçınılmış olması.

12. Overall Limitations

Gerçekçi olmayan uygulama (deployment) hedefleri nedeniyle geliştirme sürecinin tıkanması. Takım içi versiyon kontrol (Git) disiplininin zayıf olması ve karmaşık commit geçmişi.

13. Recommendation

☐ Accept ☒ **Weak Accept** ☐ Borderline ☐ Weak Reject
☐ Reject

Tavsiye: Mimari tasarımınız ve örüntü justification'larınız (gerekçeleriniz) harika. Ancak tam bir uygulama derlemeye çalışırken boğulmak yerine, mimarinin çalıştığını

kanıtlayan lokal bir prototipe geçiş yapın. Ayrıca düzensiz olan commit geçmişinizi toparlamak için standart bir commit formatı benimseyin.

14. Justification

Auditor perspektifinden bakıldığında; mimari kararlar açıkça gerekçelendirilmiş, seçilen tasarım örüntüleri sistem hedefleriyle uyuşmuş ve gereksinimlerden tasarıma kadar kusursuz bir izlenebilirlik sağlanmıştır. Mantık ve tutarlılık açısından dokümantasyon çok güçlüdür. Ancak toplantı gözlemlerine göre versiyon kontrolündeki (commitler) düzensizlik ve tüm platformlara dağıtım (deployment) yapma zorluğu, uygulamanın kalitesini düşürmektedir. Lokal bir prototipe odaklanılması yönündeki tavsiyeyle birlikte proje "Weak Accept" seviyesinde onaylanabilir.